PROGRAMMING FUNDAMENTALS

Spring 2024

3rd April

Program structure

- Programmers break down their programs into smaller units
- These smaller units are modules
- Programmers also refer to them as subroutines, procedures, functions, or methods
- The name usually reflects the programming language being used
- For example, Visual Basic programmers use procedure (or subprocedure)
- C and C++ programmers call their modules functions

Running Functions..

- The main program executes a module by calling it
- To call a module → use its name to cause the module to execute
- When the module's tasks are complete, control returns to the spot from which the module was called in the main program
- When you access a module → you pause your primary action → take care of some other task and then return to the main task exactly where you left off

Why modular approach?

- The process of breaking down a large program into modules → modularization
- Also called → functional decomposition
- Main reasons for doing so:
- Miniature programs which are combined to form a large program
- Allows complicated programs to be divided into manageable pieces
- Modularization allows multiple programmers to work on a problem
- Modularization allows you to reuse work more easily.

Functions in C++

- Like in Algebra
- Each function has a name
- It does some work
- It returns a value
  - Predefined
  - User defined

Predefined functions

- Organized in separate libraries
- E.g. I/O functions in iostream header file
- Must use include statement to be able to access these functions

Function Declaration

- Many programmers prefer to put the main( ) function as the first function in the program
- The compiler must know about a function before it is called in a program

Function Prototype

- The method used for letting the compiler know about the function without including the actual code for the function → a *function prototype*
- →Function declarations may be either function prototypes or *function definitions*

- A function prototype is a declaration without the body of the function
- A function definition is a declaration including the body of the function

Prototype example

```
int my_sum(int num1, int num2);
```

Syntax : definition

```
// function definition
string my_compare(float num1, float num2)
{
        string my_state;
        if (num1 > num2)
        my_state = "True";
    else
        my_state = "False";
return my_state;
}
```

Syntax : value returning Function Call

```
functionName(actual parameter list)
```

Parameters

- Formal : variable(s) declared in the heading → can be empty
- Actual : Variable or expression listed in a call statement→ call will also have empty parenthesis for a function with no formal parameters

return Statement

- return is a reserved word
- Function terminates and control transfers back to the call statement

- E.g. return 0; → program terminates

Example -1 -Prototype

```
#include<iostream>
 using namespace std;
 int my_sum(int num1, int num2);
 int main()
{
```

Function Definition

```
// function definition
int my_sum(int num1, int num2)
{
return num1 + num2;
}
```

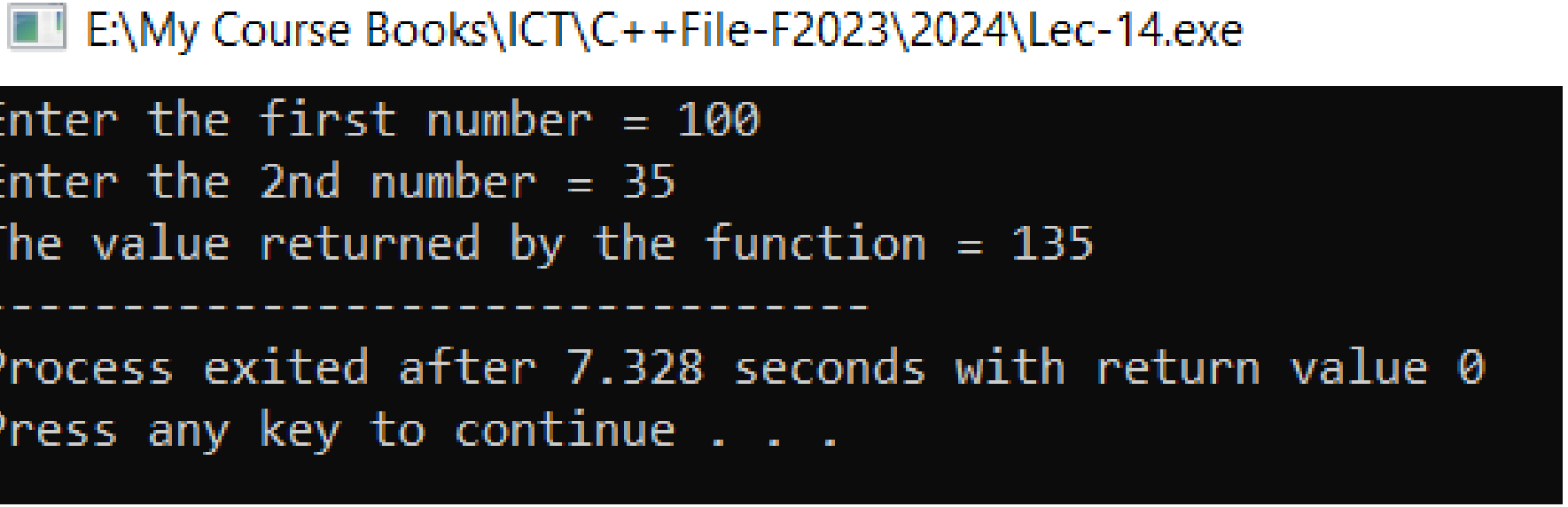
Ex-1 – Call in an assignment statement

```
int main()
{
    int n1;
    int n2;
    int sum;
    cout << "Enter first integer: ";
    cin >> n1;
    cout << "Enter 2nd integers: ";
    cin >> n2;
    // call the function in an assignment expression
    sum = my_sum(n1,n2);
    cout<<"SUM = "<<sum<<endl;
```

Ex-1 – Call in a cout statement using constant values

```
#include <iostream>
using namespace std;
int mysum(int num1, int num2);
int main()
{
    int n1,n2;
    cout<<"Enter the first number = ";
    cin>>n1;
    cout<<"Enter the 2nd number = ";
    cin>>n2;
    cout<<"The value returned by the function = "<<mysum(n1,n2);
    return 0;
}

int mysum(int num1, int num2)
{
    return num1 + num2;
}
```


E:\My Course Books\ICT\C++File-F2023\2024\Lec-14.exe
Enter the first number = 100
Enter the 2nd number = 35
The value returned by the function = 135

Process exited after 7.328 seconds with return value 0
Press any key to continue . . .

Ex-1 - Call using expressions

```
int main()
{
    int n1;
    int n2;
    int sum;
    cout << "Enter first integer: ";
    cin >> n1;
    cout << "Enter 2nd integers: ";
    cin >> n2;
    // call the function in a cout sending expressions as arguments
    cout << my_sum(2 * 3 + 5, 50 - 10)<<endl;
}
```

Ex-2

- Write a value returning function that receives two floating point numbers and returns true if the first formal parameter is greater than the second

Ex-3

- Write a function that is called by main() function.
- It accepts three integers and returns product of these three integers.

Ex-4

- Write a function that implements the switch case structure. An integer value is accepted in the main() function and used to call the function.

1 – display the name of day for today

2 – display today's date

3 – display the date as per Ramadan calendar

Otherwise – display "invalid input"

Ex-5

- Write a function that displays the received integer and adds 10 to it before displaying it.

Call the function in a loop 5 times with 5 different values.

Book Reference

- Tony Gaddis
- Ch 6